BreaQ 2026 — Quantum Dice: Random or Rigged?

Challenge At A Glance

Format: Open-ended quantum-computing security mini-project
Team size: 1–3 participants
Suggested effort: 4–6 hours for a small project; extensions are optional
Prerequisites: Basic Python; a one-qubit introduction is provided below
Deliverable: A reproducible notebook or small Python project and a short presentation
Score: 50 points

This challenge is completely independent. You may start it immediately, even if you have not attempted or completed either of the other challenges. No earlier flags, solutions, artifacts, or code are required.

Your Mission

A fictional quantum-computing startup advertises a quantum dice service:

> "Our bit generator uses a quantum circuit. Therefore, its output is fair, independent, and safe for security applications."

Your team has been asked to investigate that claim.

Build a small local model, introduce one plausible fault or bounded attack, and create a tool or experiment that exposes its effect. Then recommend what the service should do about it.

You choose the research question, failure mode, experiments, and implementation. There is no hidden password or single intended solution. A carefully explained small experiment is more valuable than a large unfinished application.

The central question is:

> **What does your evidence establish—and what does it leave unproven?**

A Quantum Starting Point

The baseline uses one qubit:

1. Initialize the qubit in state $|0\rangle$.
2. Apply a Hadamard gate, preparing $|+\rangle$.
3. Measure in the computational basis.
4. Repeat the preparation and measurement to collect a sequence of bits.

```
|0> — H — Measure
```

The ideal measurement probabilities are equal for 0 and 1. A finite sample does **not** have to contain exactly the same number of zeros and ones.

You may use Qiskit to construct the circuit and `Statevector.sample_memory` to sample its measurement distribution. This method preserves the order of sampled outcomes and does not modify the statevector; repeated samples represent repeated preparations and measurements, not repeated measurements of a single collapsed qubit. See the Qiskit Statevector documentation.

Simulation boundary: The classical simulator samples using a pseudorandom number generator. You are modeling a quantum-bit source, not generating physical quantum entropy. Do not use the resulting bits for passwords, cryptographic keys, or other security-sensitive purposes.

No real hardware, cloud account, or paid service is needed. Qiskit and ordinary Python analysis libraries are enough; Aer and additional quantum algorithms are optional, not requirements.

Choose A Research Direction

Choose **one** main direction below, or propose your own comparably small investigation.

Direction	Research question	Possible controlled fault	Possible artifact
The biased coin	Does a preparation error make one outcome more likely?	Change the state-preparation angle	Bias alarm and sensitivity study
The repeated batch	Does each request return fresh samples?	Reinitialize the simulator with the same seed on every request	Cross-request replay detector
The sticky stream	Can balanced output contain predictable patterns?	Duplicate some samples or reuse an earlier output block	Sequence-aware audit notebook
The drifting device	Can pooled statistics hide a changing source?	Change the preparation parameter at a request boundary	Sliding-window monitor
The debiasing investigation	Does post-processing fix the defect you modeled?	Compare an independently biased source with a correlated source	Assumption-aware processing experiment

Beginner starting points: The biased coin and the repeated batch need only a small source model and a focused test. You do not need to derive a full hardware model.

Label your fault honestly. Duplicating bits is a mock classical service-layer defect, not a physical quantum effect. Changing state preparation is a different mechanism. Either can be a valid project when its scope is explicit.

You may study an accidental failure instead of malicious behavior. Explain its security consequences without claiming that an anomaly proves deliberate attack.

Define Your Question

Before coding, complete this sentence:

> "We will investigate whether _____ can cause _____, and whether _____ can detect it."

Then describe your model:

Item	What to write
Protected property	Fairness, freshness, resistance to a specified predictor, or another concrete property
Adversary capability or fault	Exactly what can change and where
Trusted components	What you assume remains correct
Auditor access	Bits only, request boundaries, configuration, source code, or another declared interface
Exclusions	What your model does not cover

Choose one bounded capability. Having access to the project's Python files does not mean your modeled adversary controls every component.

Build Your Investigation

Establish A Clean Baseline

Implement the clean one-qubit model and explain its expected behavior. Collect several batches and preserve both **sample order** and **request boundaries**.

A histogram contains total counts, but not the order in which outcomes appeared. Keep the original ordered samples if you want to study adjacent bits, runs, repeated blocks, or replay across requests.

Use seeds to make experiments reproducible. Avoid restarting every clean request from the same seed: if your project studies that behavior, it belongs in the faulty model. Use fresh pseudorandom draws for different evaluation batches, for example by retaining a generator's state between requests or by using a documented seed schedule. Distinct seeds alone do not prove physical independence.

Introduce One Fault

Implement your fault as a separate function or clearly labeled option. Keep the clean and faulty interfaces similar so the auditor can evaluate either.

Before running the experiment, write a prediction:

> "This fault should affect _____, while _____ might still look normal."

If the prediction is wrong, investigate it. A well-explained negative result is valid evidence—not a reason to hide the result.

Design An Auditor

Select checks that address your research question. You might investigate:

- The fraction of ones.
- Repeated output across requests.
- Runs or transitions between adjacent bits.
- Changes across time windows.
- A statistical test with assumptions you can explain.

You do **not** need every check. Explain why yours is appropriate and what it could miss. Merely printing a statistical test's p-value without explaining the question, assumptions, and decision rule is not a complete audit.

A useful research lead is NIST's Recommendation for the Entropy Sources Used for Random Bit Generation, which discusses health tests such as repetition-count and adaptive-proportion tests. You are not expected to implement the standard or claim compliance.

Evaluate On Fresh Data

Use some batches to develop your checks or choose thresholds, then use **fresh batches** to evaluate them. Testing repeatedly on the same bits is not a new experiment.

Include both clean and faulty batches. Report:

- How many batches and bits you evaluated.
- How often clean batches raised an alarm: false alarms.
- How often faulty batches raised an alarm: detections.
- Which faulty cases were missed.
- Any choices that were tuned after seeing results.

For a small project, a clear table is sufficient. Avoid claiming universal detection from a few examples. If you test many windows or thresholds, discuss the possibility of additional false alarms.

Recommend A Response

Explain what should happen after your tool detects a problem. You could propose rejecting a batch, raising an alert, correcting a seed-lifecycle bug, monitoring drift, or investigating an appropriate post-processing method.

Detection is not repair. An alert does not make previously generated bits safe. If you investigate debiasing, identify the assumptions behind the method rather than treating it as a universal fix. For example, the usual von Neumann debiasing argument assumes independent, identically distributed input bits; arbitrary correlations require separate analysis.

Minimum Viable Submission

A complete beginner project includes:

- One clean simulated quantum-bit source.
- One explicitly modeled fault or bounded attack.
- One audit function with an interpretable result.
- Several clean and faulty evaluation batches, separate from threshold tuning.
- At least one useful figure or table based on your actual experiment outputs.
- A proposed or implemented response.
- A clear statement of limitations.

A notebook alone is enough. You do not need a dashboard, website, full statistical test suite, extra quantum algorithm, or real-device access.

A detector that misses an attack can still be an informative result if you show why, characterize its limits, and support your conclusions with controls.

Optional Extensions

Choose extensions only after your core investigation works.

- Compare detection at different sample sizes or fault strengths.
- Build a balanced stream that defeats a frequency-only audit.
- Investigate gradual drift rather than a sudden change.
- Compare two audit approaches and explain their trade-offs.
- Evaluate post-processing on both independent and correlated sources.
- Study whether someone who knows the simulator seed can reproduce apparently ordinary output.
- Explore a two-qubit model in which an additional system carries information about the output.

For the last option, distinguish the model from a real-device compromise or a device-independent security certification. Relevant side information can affect predictability even when output looks statistically ordinary; see ETSI's discussion of QRNG limitations.

Work As A Team

These responsibilities are suggestions, not fixed roles:

Responsibility	Main contribution
Source engineer	Clean circuit, controlled fault, reproducible sampling
Auditor	Checks, thresholds, clean and faulty evaluation
Research and validation lead	Sources, controls, limitations, presentation

Share your findings as you work. Every member should be able to explain the clean model, the fault, and at least one limitation. One- and two-person teams use the same minimum project scope.

Submit Your Work

Provide:

- A notebook or small Python project that runs from a fresh kernel or process.
- Dependency information and brief execution instructions.
- Your research question and threat model.
- Actual experimental results, with labeled figures or tables.
- Your conclusions, response, and limitations.
- At least two relevant sources with readable links or full reference details.

Distinguish your own work from reused examples, helper code, and external assistance. Be prepared to explain every central claim and design decision.

Present Your Findings

Plan approximately **four minutes** for this project within your final presentation, followed by a short question.

Cover:

1. Your question and threat model.
2. The clean source and the controlled fault.
3. Your most informative experiment.
4. Auditor performance, including clean controls.
5. Recommended response and limitations.

You may be asked to predict what happens when a sample size, fault strength, seed, or input stream changes. Understanding matters more than polished slides.

This project is graded independently. Completing either of the other challenges is not an eligibility requirement.

Evaluation

Category	Points	What the jury looks for
Question and bounded threat model	8	A clear property, capability or fault, access model, and exclusions
Correct quantum baseline	7	An explainable one-qubit model with justified expected behavior
Controlled fault experiment	8	A reproducible mechanism, baseline comparison, and explicit model boundaries

Category	Points	What the jury looks for
Audit and evidence	12	Appropriate checks, fresh evaluation data, clean controls, false alarms, and misses
Response and limitations	7	An appropriate recommendation and no unsupported security claims
Reproducibility and oral understanding	8	A runnable artifact, cited sources, and clear reasoning about a new case
Total	**50**	Complexity and visual polish earn no separate points

Keep Your Claims Honest

Appropriate conclusions include:

- “Our check detected repeated batches in the evaluated service requests.”
- “At the tested sample sizes, this monitor detected the preparation change in our model.”
- “This fault passed our frequency check but failed a sequence-aware check.”
- “Our method did not detect this case; the following experiment explains why.”

Avoid unsupported claims such as:

- “Passing our checks proves the output is quantum.”
- “These simulated bits are cryptographically secure.”
- “An approximately 50/50 histogram proves independence.”
- “A detected anomaly proves a malicious attacker was present.”

NIST cautions that statistical tests cannot certify a generator's suitability for cryptographic applications; see its discussion of statistical testing and cryptanalysis. In this challenge, the goal is a reproducible investigation with a defensible conclusion—not a claim of certified randomness.